```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//
// Update 9/15/2021
// Update 9/10/2020
// Updated 8/10/2019 Lab 3
// Updated 8/12/2018 V2.lab5c
// Create Date: 10/1/2015 V1.0
// Design Name:
// Module Name: top_level
//
//////////////////////////////////////////////////////////////////////////////////
`default_nettype none

module top_level(

   input wire clk_100mhz,
   input wire [15:0] sw,
   input wire btnc, btnu, btnl, btnr, btnd,
   output logic[3:0] vga_r,
   output logic[3:0] vga_b,
   output logic[3:0] vga_g,
   output logic vga_hs,
   output logic vga_vs,
   output logic led16_b, led16_g, led16_r,
   output logic led17_b, led17_g, led17_r,
   output logic [15:0] led,
   output logic ca, cb, cc, cd, ce, cf, cg, dp,  // segments a-g, dp
   output logic[7:0] an    // Display location 0-7
   );

    /*sw usage in lab03:
        * sw[1:0]: used to select between four modes:
            * 00,11: pong game
            * 01: 1 pixel white outline
            * 10: color bar
        * sw[8]: deathstar image on (1) or off (0)
        * sw[11:9]: alpha blending amount
        * sw[15:12]: puck speed
     */


    logic clk_65mhz; //65 MHz clock!
    // create 65mhz system clock, happens to match 1024 x 768 XVGA timing
    clk_wiz_lab3 clkdivider(.clk_in1(clk_100mhz), .clk_out1(clk_65mhz));

    logic [31:0] data;      //  instantiate 7-segment display; display (8) 4-bit hex
    logic [6:0] segments;
    logic [31:0] clock;
    assign {cg, cf, ce, cd, cc, cb, ca} = segments[6:0];
    display_8hex display(.clk_in(clk_65mhz),.data_in(data), .seg_out(segments), .strob
e_out(an));

    //assign seg[6:0] = segments;
    assign  dp = 1'b1;  // turn off the period

    assign led = sw;                        // turn leds on
    assign data = clock;   // display 0123456 + sw[3:0]
    assign led16_r = btnl;                  // left button -> red led
    assign led16_g = btnc;                  // center button -> green led
    assign led16_b = btnr;                  // right button -> blue led
    assign led17_r = btnl;
    assign led17_g = btnc;
    assign led17_b = btnr;
```

```
    logic [10:0] hcount;    // pixel on current line
    logic [9:0] vcount;     // line number
    logic hsync, vsync, blank; //control signals for vga
    logic [11:0] pixel;
    logic [11:0] rgb;

    xvga xvga1(.vclock_in(clk_65mhz),.hcount_out(hcount),.vcount_out(vcount),
          .hsync_out(hsync),.vsync_out(vsync),.blank_out(blank));

    // btnc button is user reset
    logic reset;
    debounce db1(.reset_in(reset),.clock_in(clk_65mhz),.noisy_in(btnc),.clean_out(rese
t));

    // UP and DOWN buttons for pong paddle
    logic up,down;
    debounce db2(.reset_in(reset),.clock_in(clk_65mhz),.noisy_in(btnu),.clean_out(up))
;
    debounce db3(.reset_in(reset),.clock_in(clk_65mhz),.noisy_in(btnd),.clean_out(down
));

    // UP and DOWN buttons for pong paddle
    logic up2,down2;
    debounce db4(.reset_in(reset),.clock_in(clk_65mhz),.noisy_in(btnl),.clean_out(up2)
);
    debounce db5(.reset_in(reset),.clock_in(clk_65mhz),.noisy_in(btnr),.clean_out(down
2));

    logic game_end;

    logic phsync,pvsync,pblank;
    pong_game pg(.vclock_in(clk_65mhz),.reset_in(reset),
                 .up_in(up),.down_in(down),.up_in2(up2),.down_in2(down2), .pspeed_in(sw
[15:12]),
                 .hcount_in(hcount),.vcount_in(vcount),
                 .hsync_in(hsync),.vsync_in(vsync),.blank_in(blank),
                 .alpha_in(sw[11:9]), .image_set_in(sw[8]),
                 .phsync_out(phsync),.pvsync_out(pvsync),.pblank_out(pblank),.pixel_out
(pixel), .game_end_out(game_end));


    logic border = (hcount==0 | hcount==1023 | vcount==0 | vcount==767 |
                    hcount == 512 | vcount == 384);

    logic b,hs,vs;

    logic [23:0] count;
    logic [31:0] clock;
    always_ff @(posedge clk_65mhz) begin
        if (reset) begin
            clock <= 0;
            count <= 24'd0;
        end else begin
            if(count >= 24'd6_500_000) begin
                count <= 24'd0;
                clock <= clock + 1;
            end else if (game_end) begin
                count <= count;
            end else begin
                count <= count + 24'd1;
            end
        end

        if (sw[1:0] == 2'b01) begin
             // 1 pixel outline of visible area (white)
```

```
            hs <= hsync;
            vs <= vsync;
            b <= blank;
            rgb <= {12{border}};
        end else if (sw[1:0] == 2'b10) begin
            // color bars
            hs <= hsync;
            vs <= vsync;
            b <= blank;
            rgb <= {{4{hcount[8]}}, {4{hcount[7]}}, {4{hcount[6]}}} ;
         end else begin
            // default: pong
            hs <= phsync;
            vs <= pvsync;
            b <= pblank;
            rgb <= pixel;
         end
    end

//    assign rgb = sw[0] ? {12{border}} : pixel ; //{{4{hcount[7]}}, {4{hcount[6]}}, {
4{hcount[5]}}};

    // the following lines are required for the Nexys4 VGA circuit - do not change
    assign vga_r = ~b ? rgb[11:8]: 0;
    assign vga_g = ~b ? rgb[7:4] : 0;
    assign vga_b = ~b ? rgb[3:0] : 0;

    assign vga_hs = ~hs;
    assign vga_vs = ~vs;
//ila_0  myila(.clk(clk_65mhz),.probe0(hsync),.probe1(hcount),.probe2(pixel));
endmodule

////////////////////////////////////////////////////////////////////////////////
//
// pong_game: the game itself!
//
////////////////////////////////////////////////////////////////////////////////

module pong_game (
   input wire vclock_in,        // 65MHz clock
   input wire reset_in,         // 1 to initialize module
   input wire up_in,            // 1 when paddle should move up
   input wire down_in,          // 1 when paddle should move down

   input wire up_in2,
   input wire down_in2,

   input wire [3:0] pspeed_in,  // puck speed in pixels/tick
   input wire [10:0] hcount_in, // horizontal index of current pixel (0..1023)
   input wire [9:0]  vcount_in, // vertical index of current pixel (0..767)
   input wire hsync_in,         // XVGA horizontal sync signal (active low)
   input wire vsync_in,         // XVGA vertical sync signal (active low)
   input wire blank_in,         // XVGA blanking (1 means output black pixel)
   input wire [2:0] alpha_in,      //alpha flavor,
   input wire image_set_in,     //when 1, puck becomes image, when 0, puck is plain bo
x

   output logic phsync_out,       // pong game's horizontal sync
   output logic pvsync_out,       // pong game's vertical sync
   output logic pblank_out,       // pong game's blanking
   output logic [11:0] pixel_out,  // pong game's pixel  // r=11:8, g=7:4, b=3:0

   output logic game_end_out
   );
     parameter S_WIDTH = 1024;
```

```
     parameter S_HEIGHT = 768;
    //LOGIC VARIABLES
    //paddle 1
    logic [10:0] paddle1_x = 11'd6;
    logic [9:0] paddle1_y = 10'd343;
    logic [11:0] paddle1_pixel;
    parameter paddle1_width = 16;
    parameter paddle1_height = 128;
    //paddle 2
    logic [10:0] paddle2_x = 11'd1023 - 11'd6 - 11'd16;
    logic [9:0] paddle2_y = 10'd343;
    logic [11:0] paddle2_pixel;
    parameter paddle2_width = 16;
    parameter paddle2_height = 128;
    //puck
    logic [10:0] puck1_x;
    logic [9:0] puck1_y;
    logic [11:0] puck1_pixel;
    logic puck1_direction_x = 1'b1;
    logic puck1_direction_y = 1'b1;
    parameter puck1_width = 256;
    parameter puck1_height = 240;
    //center blob
    logic [10:0] obs1_x = 11'd512;
    logic [9:0] obs1_y = 10'd384;
    logic [11:0] obs1_pixel;
    parameter obs1_width = 64;
    parameter obs1_height = 64;
    //death star
    logic [11:0] death_star1_pixel;
    //alpha
    logic [7:0] alpha;
    logic [11:0] alpha_pixel;
    logic [5:0] alpha_pixel_r;
    logic [5:0] alpha_pixel_g;
    logic [5:0] alpha_pixel_b;
    //miscellenous
    logic [11:0] puck1_choice_pixel;


    //instantiations
    blob #(.WIDTH(paddle1_width),.HEIGHT(paddle1_height),.COLOR(12'hF0F)) paddle1(.x_i
n(paddle1_x),.y_in(paddle1_y),.hcount_in(hcount_in),.vcount_in(vcount_in),.pixel_out(p
addle1_pixel));
    blob #(.WIDTH(paddle2_width),.HEIGHT(paddle2_height),.COLOR(12'hF0F)) paddle2(.x_i
n(paddle2_x),.y_in(paddle2_y),.hcount_in(hcount_in),.vcount_in(vcount_in),.pixel_out(p
addle2_pixel));

    blob #(.WIDTH(puck1_width),.HEIGHT(puck1_height),.COLOR(12'hFFF)) puck1(.x_in(puck
1_x),.y_in(puck1_y),.hcount_in(hcount_in),.vcount_in(vcount_in),.pixel_out(puck1_pixel
));
    blob #(.WIDTH(obs1_width),.HEIGHT(obs1_height),.COLOR(12'hF00)) obs1(.x_in(obs1_x)
,.y_in(obs1_y),.hcount_in(hcount_in),.vcount_in(vcount_in),.pixel_out(obs1_pixel));
    picture_blob #(.WIDTH(puck1_width),.HEIGHT(puck1_height)) death_star1(.pixel_clk_i
n(vclock_in), .x_in(puck1_x),.y_in(puck1_y),.hcount_in(hcount_in),.vcount_in(vcount_in
),.pixel_out(death_star1_pixel));

    always_comb begin
        case(alpha_in)
            3'b000 : alpha = 8'd0;
            3'b001 : alpha = 8'd1;
            3'b010 : alpha = 8'd2;
            3'b011 : alpha = 8'd3;
            3'b100 : alpha = 8'd4;
            default : alpha = 8'd4;
```

```
        endcase
        puck1_choice_pixel =  ((image_set_in) ? death_star1_pixel : puck1_pixel);

        alpha_pixel_r = (puck1_choice_pixel[11:8]*alpha + obs1_pixel[11:8]*(4 - alpha)
) >> 2;
        alpha_pixel_g = (puck1_choice_pixel[7:4]*alpha + obs1_pixel[7:4]*(4 - alpha))
>> 2;
        alpha_pixel_b = (puck1_choice_pixel[3:0]*alpha + obs1_pixel[3:0]*(4 - alpha))
>> 2;

        alpha_pixel = {alpha_pixel_r[3:0], alpha_pixel_g[3:0], alpha_pixel_b[3:0]};
        if(obs1_pixel !=0 && puck1_choice_pixel !=0) begin
            pixel_out = paddle1_pixel + paddle2_pixel + alpha_pixel;
        end else begin
            pixel_out = paddle1_pixel + paddle2_pixel + obs1_pixel + puck1_choice_pixe
l;
        end
    end
   always_ff @(posedge vclock_in) begin
        phsync_out <= hsync_in;
        pvsync_out <= vsync_in;
        pblank_out <= blank_in;
        if(reset_in) begin
            puck1_x <= S_WIDTH >> 1;
            puck1_y <= S_HEIGHT >> 1;
            puck1_direction_x <= 1'b1;
            puck1_direction_y <= 1'b1;

            paddle1_y <= S_HEIGHT >> 1;
            paddle2_y <= S_HEIGHT >> 1;
        end else begin
            game_end_out = (pspeed_in == 4'b0) ? 1'b1: 1'b0;
            if(vcount_in == 10'd0 && hcount_in == 11'd0) begin
            //Puck logic
                //Puck Colisions
                if ( (puck1_x <=( 11'd0 + pspeed_in)) && !puck1_direction_x) begin
                    puck1_x <= puck1_x;
                    game_end_out <= 1'b1;
                end else if ((puck1_x >= S_WIDTH - puck1_width - pspeed_in) && puck1_d
irection_x ) begin
                    puck1_x <= puck1_x - pspeed_in;
                    puck1_direction_x <= 1'b0;
                    //game_end_out <= 1'b1;
                    //
                end else if ((puck1_y <= 10'd0 + pspeed_in) && !puck1_direction_y ) be
gin
                    puck1_y <= puck1_y + pspeed_in;
                    puck1_direction_y <= 1'b1;
                end else if  ((puck1_y >= S_HEIGHT - puck1_height - pspeed_in) && puck
1_direction_y) begin
                    puck1_y <= puck1_y - pspeed_in;
                    puck1_direction_y <= 1'b0;
                end else if ((puck1_x >= paddle1_x && puck1_x <= paddle1_x + paddle1_w
idth + pspeed_in) && (puck1_y + puck1_height >= paddle1_y && puck1_y <= paddle1_y + pa
ddle1_height)) begin
                    puck1_x <= puck1_x + pspeed_in;
                    puck1_direction_x <= 1'b1;
                end else if ( (puck1_x + puck1_width <= paddle2_x + paddle2_width && p
uck1_x + puck1_width >= paddle2_x - pspeed_in) && (puck1_y + puck1_height >= paddle2_y
 && puck1_y <= paddle2_y + paddle2_height)) begin
                    puck1_x <= puck1_x - pspeed_in;
                    puck1_direction_x <= 1'b0;
                end else begin
                    if (puck1_direction_x) begin
                        puck1_x <= puck1_x + pspeed_in;
```

```
                    end else begin
                        puck1_x <= puck1_x - pspeed_in;
                    end
                    if (puck1_direction_y) begin
                        puck1_y <= puck1_y + pspeed_in;
                    end else begin
                        puck1_y <= puck1_y - pspeed_in;
                    end
                end
                //Paddle 1 Logic
                if (up_in && !(paddle1_y <= 10'd0 + 10'd4)) begin
                    paddle1_y <= paddle1_y - 10'd5;
                end
                if (down_in && !( paddle1_y >= S_HEIGHT - paddle1_height - 10'd4) ) be
gin
                    paddle1_y <= paddle1_y + 10'd5;
                end
                //Paddle 2 Logic
                if (up_in2 && !(paddle2_y <= 10'd0 + 10'd4)) begin
                    paddle2_y <= paddle2_y - 10'd5;
                end
                if (down_in2 && !( paddle2_y >= S_HEIGHT - paddle2_height -10'd4) ) be
gin
                    paddle2_y <= paddle2_y + 10'd5;
                end
            end
        end
    end

endmodule

module synchronize #(parameter NSYNC = 3)  // number of sync flops.  must be >= 2
                    (input wire clk,in,
                     output logic out);

  logic [NSYNC-2:0] sync;

  always_ff @ (posedge clk) begin
    {out,sync} <= {sync[NSYNC-2:0],in};
  end
endmodule

////////////////////////////////////////////////////////////////////////////////
//
// Pushbutton Debounce Module (video version - 24 bits)
//
////////////////////////////////////////////////////////////////////////////////

module debounce (input wire reset_in, clock_in, noisy_in,
                 output logic clean_out);

    logic [19:0] count;
    logic new_input;

    always_ff @(posedge clock_in)
      if (reset_in) begin
         new_input <= noisy_in;
         clean_out <= noisy_in;
         count <= 0; end
      else if (noisy_in != new_input) begin new_input<=noisy_in; count <= 0; end
      else if (count == 1000000) clean_out <= new_input;
      else count <= count+1;


endmodule
```

```
//////////////////////////////////////////////////////////////////////////////////
// Engineer:   g.p.hom
//
// Create Date:    18:18:59 04/21/2013
// Module Name:    display_8hex
// Description:  Display 8 hex numbers on 7 segment display
//
//////////////////////////////////////////////////////////////////////////////////

module display_8hex(
    input wire clk_in,                  // system clock
    input wire [31:0] data_in,          // 8 hex numbers, msb first
    output logic [6:0] seg_out,     // seven segment display output
    output logic [7:0] strobe_out   // digit strobe
    );

    localparam bits = 13;

    logic [bits:0] counter = 0;  // clear on power up

    logic [6:0] segments[15:0]; // 16 7 bit memorys
    assign segments[0]  = 7'b100_0000;  // inverted logic
    assign segments[1]  = 7'b111_1001;  // gfedcba
    assign segments[2]  = 7'b010_0100;
    assign segments[3]  = 7'b011_0000;
    assign segments[4]  = 7'b001_1001;
    assign segments[5]  = 7'b001_0010;
    assign segments[6]  = 7'b000_0010;
    assign segments[7]  = 7'b111_1000;
    assign segments[8]  = 7'b000_0000;
    assign segments[9]  = 7'b001_1000;
    assign segments[10] = 7'b000_1000;
    assign segments[11] = 7'b000_0011;
    assign segments[12] = 7'b010_0111;
    assign segments[13] = 7'b010_0001;
    assign segments[14] = 7'b000_0110;
    assign segments[15] = 7'b000_1110;

    always_ff @(posedge clk_in) begin
      // Here I am using a counter and select 3 bits which provides
      // a reasonable refresh rate starting the left most digit
      // and moving left.
      counter <= counter + 1;
      case (counter[bits:bits-2])
          3'b000: begin  // use the MSB 4 bits
                  seg_out <= segments[data_in[31:28]];
                  strobe_out <= 8'b0111_1111 ;
                end

          3'b001: begin
                  seg_out <= segments[data_in[27:24]];
                  strobe_out <= 8'b1011_1111 ;
                end

          3'b010: begin
                   seg_out <= segments[data_in[23:20]];
                   strobe_out <= 8'b1101_1111 ;
                  end
          3'b011: begin
                  seg_out <= segments[data_in[19:16]];
                  strobe_out <= 8'b1110_1111;
                end
          3'b100: begin
                  seg_out <= segments[data_in[15:12]];
```

```
                strobe_out <= 8'b1111_0111;
               end

          3'b101: begin
                  seg_out <= segments[data_in[11:8]];
                  strobe_out <= 8'b1111_1011;
                 end

          3'b110: begin
                   seg_out <= segments[data_in[7:4]];
                   strobe_out <= 8'b1111_1101;
                  end
          3'b111: begin
                  seg_out <= segments[data_in[3:0]];
                  strobe_out <= 8'b1111_1110;
                 end

       endcase
      end

endmodule

//////////////////////////////////////////////////////////////////////////////////
// Update: 8/8/2019 GH
// Create Date: 10/02/2015 02:05:19 AM
// Module Name: xvga
//
// xvga: Generate VGA display signals (1024 x 768 @ 60Hz)
//
//                              ---- HORIZONTAL -----     ------VERTICAL -----
//                              Active                    Active
//                    Freq      Video   FP  Sync   BP      Video   FP  Sync  BP
//   640x480, 60Hz    25.175    640     16    96   48       480    11   2    31
//   800x600, 60Hz    40.000    800     40   128   88       600     1   4    23
//   1024x768, 60Hz   65.000    1024    24   136  160       768     3   6    29
//   1280x1024, 60Hz  108.00    1280    48   112  248       768     1   3    38
//   1280x720p 60Hz   75.25     1280    72    80  216       720     3   5    30
//   1920x1080 60Hz   148.5     1920    88    44  148      1080     4   5    36
//
// change the clock frequency, front porches, sync's, and back porches to create
// other screen resolutions
//////////////////////////////////////////////////////////////////////////////////

module xvga(input wire vclock_in,
            output logic [10:0] hcount_out,    // pixel number on current line
            output logic [9:0] vcount_out,     // line number
            output logic vsync_out, hsync_out,
            output logic blank_out);

   parameter DISPLAY_WIDTH  = 1024;       // display width
   parameter DISPLAY_HEIGHT = 768;        // number of lines

   parameter  H_FP = 24;                  // horizontal front porch
   parameter  H_SYNC_PULSE = 136;         // horizontal sync
   parameter  H_BP = 160;                 // horizontal back porch

   parameter  V_FP = 3;                   // vertical front porch
   parameter  V_SYNC_PULSE = 6;           // vertical sync
   parameter  V_BP = 29;                  // vertical back porch

   // horizontal: 1344 pixels total
   // display 1024 pixels per line
   logic hblank,vblank;
   logic hsyncon,hsyncoff,hreset,hblankon;
   assign hblankon = (hcount_out == (DISPLAY_WIDTH -1));
```

```
   assign hsyncon = (hcount_out == (DISPLAY_WIDTH + H_FP - 1));  //1047
   assign hsyncoff = (hcount_out == (DISPLAY_WIDTH + H_FP + H_SYNC_PULSE - 1));  // 11
83
   assign hreset = (hcount_out == (DISPLAY_WIDTH + H_FP + H_SYNC_PULSE + H_BP - 1));
//1343

   // vertical: 806 lines total
   // display 768 lines
   logic vsyncon,vsyncoff,vreset,vblankon;
   assign vblankon = hreset & (vcount_out == (DISPLAY_HEIGHT - 1));   // 767
   assign vsyncon = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP - 1));  // 771
   assign vsyncoff = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP + V_SYNC_PULSE - 1
));  // 777
   assign vreset = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP + V_SYNC_PULSE + V_B
P - 1)); // 805

   // sync and blanking
   logic next_hblank,next_vblank;
   assign next_hblank = hreset ? 0 : hblankon ? 1 : hblank;
   assign next_vblank = vreset ? 0 : vblankon ? 1 : vblank;
   always_ff @(posedge vclock_in) begin
      hcount_out <= hreset ? 0 : hcount_out + 1;
      hblank <= next_hblank;
      hsync_out <= hsyncon ? 0 : hsyncoff ? 1 : hsync_out;  // active low

      vcount_out <= hreset ? (vreset ? 0 : vcount_out + 1) : vcount_out;
      vblank <= next_vblank;
      vsync_out <= vsyncon ? 0 : vsyncoff ? 1 : vsync_out;  // active low

      blank_out <= next_vblank | (next_hblank & ~hreset);
   end
endmodule

module blob
   #(parameter WIDTH = 64,            // default width: 64 pixels
               HEIGHT = 64,           // default height: 64 pixels
               COLOR = 12'hFFF)  // default color: white
   (input wire [10:0] x_in,hcount_in,
    input wire [9:0] y_in,vcount_in,
    output logic [11:0] pixel_out);

   always_comb begin
      if  ((hcount_in >= x_in && hcount_in < (x_in+WIDTH)) &&
           (vcount_in >= y_in && vcount_in < (y_in+HEIGHT)))
        pixel_out = COLOR;
      else
        pixel_out = 0;
   end
endmodule

module picture_blob
   #(parameter WIDTH = 256,     // default picture width
               HEIGHT = 240)    // default picture height
   (input wire pixel_clk_in,
    input wire [10:0] x_in,hcount_in,
    input wire [9:0] y_in,vcount_in,
    output logic [11:0] pixel_out);

   logic [15:0] image_addr;   // num of bits for 256*240 ROM
   logic [7:0] image_bits, red_mapped, green_mapped, blue_mapped;

   // calculate rom address and read the location
   assign image_addr = (hcount_in-x_in) + (vcount_in-y_in) * WIDTH;
   image_rom  rom1(.clka(pixel_clk_in), .addra(image_addr), .douta(image_bits));
```

```
   // use color map to create 4 bits R, 4 bits G, 4 bits B
   // since the image is greyscale, just replicate the red pixels
   // and not bother with the other two color maps.
   red_coe rcm (.clka(pixel_clk_in), .addra(image_bits), .douta(red_mapped));
   //green_coe gcm (.clka(pixel_clk_in), .addra(image_bits), .douta(green_mapped));
   //blue_coe bcm (.clka(pixel_clk_in), .addra(image_bits), .douta(blue_mapped));
   // note the one clock cycle delay in pixel!
   always_ff @ (posedge pixel_clk_in) begin
     if ((hcount_in >= x_in && hcount_in < (x_in+WIDTH)) &&
          (vcount_in >= y_in && vcount_in < (y_in+HEIGHT)))
        // use MSB 4 bits
        pixel_out <= {red_mapped[7:4], red_mapped[7:4], red_mapped[7:4]}; // greyscale
        //pixel_out <= {red_mapped[7:4], 8'h0}; // only red hues
        else pixel_out <= 0;
   end
endmodule


`default_nettype wire
```